Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Otomata

Milestone 1

Semester II / 2025–2026

Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021
Nicholas Wise Saragih Sumbayak - 13524037
Kurt Mikhael Purba - 13524065
Rava Khoman Tuah Saragih - 13524107

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Introduction **3**
1.1 Background . . . 3
1.2 Problem Statement . . . 3
1.3 System Overview . . . 3
1.4 General Specifications . . . 3

2 Theoretical Background **3**
2.1 Core Algorithm / Concept A . . . 3
2.2 Core Algorithm / Concept B . . . 3
2.3 Tech Stack Overview . . . 3

3 Implementation **4**
3.1 System Architecture . . . 4
3.2 Directory Structure . . . 4
3.3 Backend . . . 4
3.3.1 Entry Point . . . 4
3.3.2 Core Algorithm Package . . . 4
3.3.3 API Endpoints . . . 5
3.4 Frontend . . . 5
3.4.1 Component Architecture . . . 5
3.4.2 State Management . . . 5
3.4.3 UI/UX Principles Applied . . . 5
3.5 Data Layer . . . 5
3.5.1 Dataset . . . 5
3.5.2 Preprocessing Pipeline . . . 6
3.6 Containerisation with Docker . . . 6
3.6.1 Multi-stage Build Strategy . . . 6
3.6.2 Running Locally . . . 6
3.7 Performance Metrics . . . 7

4 Experiments and Results **7**
4.1 Test Case Category A . . . 7
4.1.1 Test Case 1: *[Description]* . . . 7
4.1.2 Test Case 2: *[Description]* . . . 7
4.2 Test Case Category B . . . 7
4.3 Overall Analysis . . . 7
4.3.1 Algorithm A — Strengths and Limitations . . . 7
4.3.2 Algorithm B — Strengths and Limitations . . . 8

5 Conclusion **8**
5.1 Summary . . . 8
5.2 Suggestions for Future Work . . . 8
5.3 Individual Reflections . . . 8
5.3.1 Nathanael Imandatua Manurung - 13524021 . . . 8
5.3.2 Nicholas Wise Saragih Sumbayak - 13524037 . . . 8
5.3.3 Kurt Mikhael Purba - 13524065 . . . 8

References **9**

Appendix **10**

1 Introduction

1.1 Background

Replace this paragraph with the motivation and context for your project. Describe the real-world or academic problem that the system aims to solve.

1.2 Problem Statement

State the core problem clearly. Mention the input, desired output, and any key constraints.

1.3 System Overview

Milestone 1 is a ... system that ...

1.4 General Specifications

1. Feature / requirement one.
2. Feature / requirement two.
3. **Bonus:** Optional bonus feature.

2 Theoretical Background

2.1 Core Algorithm / Concept A

Describe the mathematical or algorithmic foundation. Use equations where appropriate.

$$A = U\Sigma V^\top \tag{1}$$

2.2 Core Algorithm / Concept B

...

2.3 Tech Stack Overview

Briefly justify the choice of technologies.

Layer	Technology	Rationale
Frontend	React + TypeScript	Component-based UI, type safety
Backend	Go (Golang)	High-performance, built-in concurrency
Build tool	Docker	Reproducible containerised deployment

Tabel 1: Tech stack summary

3 Implementation

3.1 System Architecture

The application follows a **client-server** architecture with a clear separation between the presentation layer (frontend) and the application/data layer (backend).

3.2 Directory Structure

Listing 1: Project directory layout

```
project-root/
    src/
        backend/          # Server-side code
            main.go
            algeo/        # Core algorithm package
            preprocess/   # Data preprocessing
            caching/      # Model persistence
        frontend/         # Client-side code
            src/
                components/
                pages/
                hooks/
    data/
        covers/           # Image assets
        txt/              # Text corpus
        mapper.json       # Metadata index
    be.Dockerfile
    fe.Dockerfile
```

3.3 Backend

3.3.1 Entry Point

3.3.2 Core Algorithm Package

Algorithm

Listing 2: Core algorithm — pseudocode / excerpt

```go
func ComputeSomething(input Matrix, k int) (*Result, error) {
    // Step 1: preprocess
    // Step 2: core computation
    // Step 3: return result
}
```

Algorithm steps:

1. Step one description.
2. Step two description.
3. Step three description.

Algorithm B ...

3.3.3 API Endpoints

Method	Endpoint	Input	Description
GET	`/items`	query params	List items with pagination
GET	`/items/:id`	path param	Item detail + recommendations
POST	`/search/image`	multipart	Visual similarity search
POST	`/search/text`	JSON body	Semantic text search

Tabel 2: REST API endpoints

3.4 Frontend

3.4.1 Component Architecture

The frontend is a Single Page Application (SPA) built with React + TypeScript. Key design choices:

- **React Router** — client-side routing with no full page reloads.
- **Context API** — global state management without external libraries.
- **Tailwind CSS** — utility-first styling for responsive layouts.

3.4.2 State Management

3.4.3 UI/UX Principles Applied

1. **Visual Hierarchy** — consistent typographic scale across all views.
2. **Responsive Design** — mobile-first breakpoints (`sm`, `md`, `lg`).
3. **Micro-interactions** — subtle animations via Framer Motion for tactile feedback.
4. **Error & Empty States** — informative messages when searches return no results.

3.5 Data Layer

3.5.1 Dataset

The dataset consists of **N** items stored as file-based records:

```
data/
    mapper.json          # ID -> metadata mapping
    covers/              # Image files (.jpg)
    txt/                 # Text content (.txt)
```

3.5.2 Preprocessing Pipeline

1. **Load** raw files from disk.
2. **Normalise** (grayscale conversion, lowercasing, tokenisation, etc.).
3. **Index** into an efficient in-memory structure.
4. **Cache** computed representations to disk via `encoding/gob` (or pickle, etc.).

3.6 Containerisation with Docker

3.6.1 Multi-stage Build Strategy

Both backend and frontend use multi-stage Dockerfiles to minimise final image size.

Listing 3: Backend Dockerfile (abbreviated)

```
# Stage 1     compile
FROM golang:1.22 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN GOOS=linux go build -o server ./main.go

# Stage 2     minimal runtime
FROM ubuntu:22.04
WORKDIR /app
COPY --from=builder /app/server .
VOLUME ["/app/data"]
EXPOSE 8080
ENTRYPOINT ["./server"]
```

3.6.2 Running Locally

```
# Backend
docker build -t myapp-backend -f be.Dockerfile .
docker run --rm -p 8080:8080 \
  -e FRONTEND_URL=http://localhost:3000 \
  -v "$(pwd)/data:/app/data:ro" \
  myapp-backend

# Frontend
docker build -t myapp-frontend \
  --build-arg VITE_API_URL=http://localhost:8080 \
  -f fe.Dockerfile .
docker run --rm -p 3000:80 myapp-frontend
```

3.7 Performance Metrics

Operation	**Time**
Document preprocessing	7.67 s
Vocabulary build	1.24 s
Term-document matrix	477 ms
SVD (50 components)	6.06 s
Model training (total)	23.0 s
Query response (P50)	< 50 ms
Query response (P99)	< 100 ms

Tabel 3: Training and inference performance (N = 523 documents)

4 Experiments and Results

4.1 Test Case Category A

4.1.1 Test Case 1: *[Description]*

Input: Describe the input used.
Expected output: What the correct result should be.

[Screenshot placeholder]

Gambar 1: Test case 1 — result screenshot

Analysis: Explain whether the result is correct and why.

4.1.2 Test Case 2: *[Description]*

...

4.2 Test Case Category B

...

4.3 Overall Analysis

4.3.1 Algorithm A — Strengths and Limitations

Strengths:

- Strength one.
- Strength two.

Limitations:

- Limitation one.
- Limitation two.

4.3.2 Algorithm B — Strengths and Limitations

...

5 Conclusion

5.1 Summary

1. **Finding 1** — description.
2. **Finding 2** — description.
3. **Finding 3** — description.

5.2 Suggestions for Future Work

1. Idea for improvement or extension one.
2. Idea for improvement or extension two.

5.3 Individual Reflections

5.3.1 Nathanael Imandatua Manurung - 13524021

Role:
Reflection: Write your personal reflection here.

5.3.2 Nicholas Wise Saragih Sumbayak - 13524037

Role:
Reflection: ...

5.3.3 Kurt Mikhael Purba - 13524065

Role:
Reflection: ...

References

[1] A. Author, *Book Title*, Edition. City: Publisher, Year.

[2] A. Author and B. Author, "Article title," *Journal Name*, vol. X, no. Y, pp. 1–10, Month Year. https://doi.org/10.xxxx/xxxxx.

[3] Organisation, "Web resource title," Year. [Online]. Available: https://example.com.

Appendix

Resource	Link
GitHub Repository	https://github.com/your-org/your-repo
Live Deployment	https://your-deployment-url.example.com

Tabel 4: Repository and deployment links